



Manual testing techniques 10m

Let's look at some more techniques which can be used when you're testing manually.

Specification-based techniques

All pairs/pairwise

Normally, it's not possible to test all combinations of input variables. The all-pairs (or pairwise) technique enables you to create a subset of test combinations that pair values, resulting in a reasonably small number of test cases which still provide 100 per cent test coverage. Equivalence partitioning and boundary value analysis can help identify the required inputs and values.

Let's look at an example that has several variables (adapted from Vijay, 2025).

A web application is designed to help potential tenants search online for houses to rent that suit their needs within their budget. So, there are six variables:

- location (CBD, suburbs)
- rent price range (>\$500 per week, <\$500 per week)
- number of bedrooms (two, three or four)
- parking (off-street or on-street)
- pets allowed (yes or no)
- number of storeys (one or two).

With five variables having two possible values and one variable having three values, this gives 96 combinations ($2 \times 2 \times 3 \times 2 \times 2 \times 2 = 96$). For the purposes of this example, we have omitted some values, like rent = \$500, which would bump up the

Skip to main content

number of required tests.

To reduce the testing time, the all-pairs technique can be applied.

With this technique, a table is created to arrange the variables and their corresponding values, putting the variable with the highest number of values first.

In the table, the variable with the highest number of values should be recorded in column 1 as many times as the number of values held by the next highest variable. In this example, the number of bedrooms is the variable with the highest number of values (three) and should be written twice (as all the other variables have two values).

Then, for each set of values in column 1, apply each value for the remaining variables in columns 2, 3 and so on.

Note that the order of values needs to be reversed once in each column.

No of bedrooms	Location	Rent price range	Parking	Pets	No of storeys
Two	CBD	>\$500 per week	Off-street	Yes	One
Two	Suburbs	<\$500 per week	On-street	No	Two
Three	Suburbs	>\$500 per week	Off-street	No	Two
Three	CBD	<\$500 per week	On-street	Yes	One
Four	CBD	>\$500 per week	On-street	No	One
Four	Suburbs	<\$500 per week	Off-street	Yes	Two

If you look at the table above, you will see two test combinations are missing:

- CBD and two storeys
- suburbs and one storey.

[Skip to main content](#)

This requires two more rows. See below.

No of bedrooms	Location	Rent price range	Parking	Pets	No of storeys
Two	CBD	>\$500 per week	Off-street	Yes	One
Two	Suburbs	<\$500 per week	On-street	No	Two
	CBD				Two
Three	Suburbs	>\$500 per week	Off-street	No	Two
Three	CBD	<\$500 per week	On-street	Yes	One
	Suburbs				One
Four	CBD	>\$500 per week	On-street	No	One
Four	Suburbs	<\$500 per week	Off-street	Yes	Two

The remaining cells in the above table can be filled with any variables because these are arbitrary.

Using this technique to design your test cases enables you to test eight combinations instead of 96.

Classification tree method

As with many test techniques, the aim of the classification tree method is to determine a sufficient but minimum number of test case specifications.

It's a graphical technique that combines the results of equivalence partitioning and boundary value analysis with an overview of the SUT.

[Skip to main content](#)

The tree starts with a single aspect (a classification) of the software you're testing. Branches are added for the classes (inputs) you may wish to test, and then leaves (values) are added to the branches.

Let's look at an example of a database system that has classifications for user privileges, operations and access methods.

The table below demonstrates the classifications, classes and values.

Classifications (test aspects) – the tree	Classes (inputs) – the branches	Values (further classes) – the leaves
User privilege	■ Regular user ■ Administrator user	
Operation	■ Add ■ Edit ■ Delete	
Access method	■ Native tool ■ Web browser ■ API	Web browser: ■ Google Chrome ■ Mozilla Firefox ■ Apple Safari

Can you see that this gives 30 possible test cases because $2 \text{ user privileges} \times 3 \text{ operations} \times 5 \text{ access methods} = 30 \text{ test cases}$?

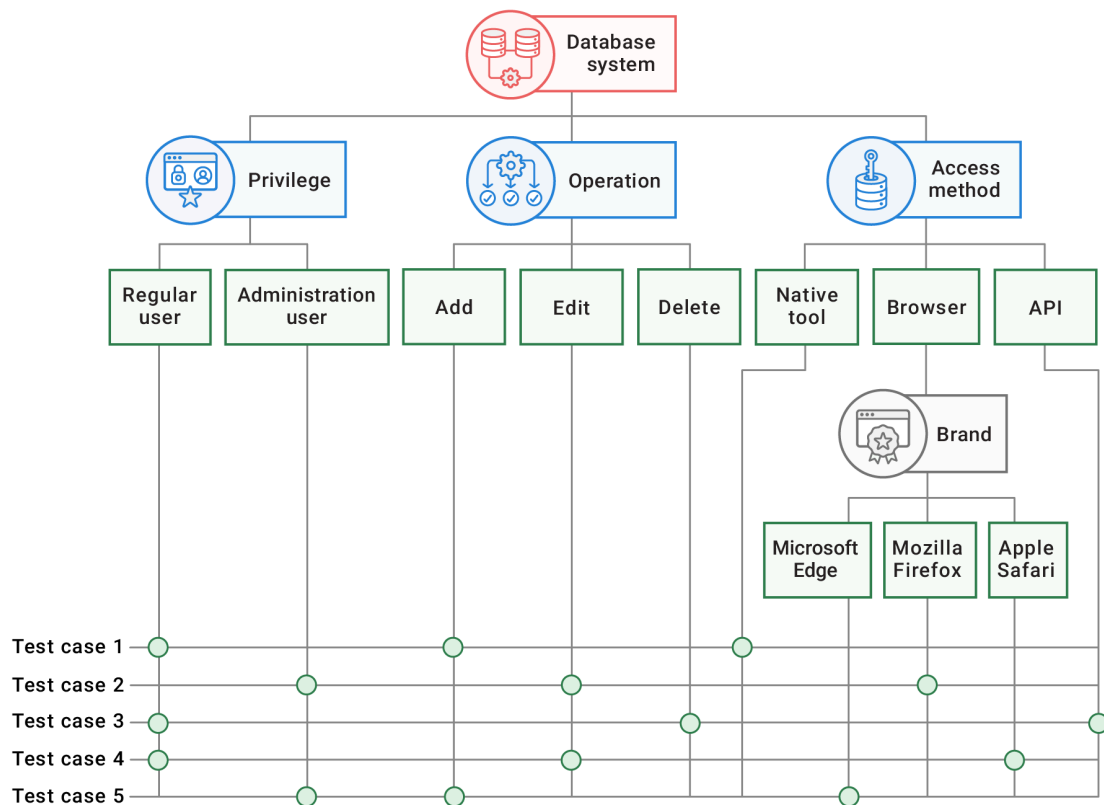
For minimum test coverage, use the classification with the highest number of classes. In this example, it's the access method that has five classes. So, five test cases are sufficient.

A classification tree can be drawn up using this information, and from it, test cases can be selected as shown at the bottom of the tree below.

[Skip to main content](#)



Classification tree



© © The Open Polytechnic of New Zealand



The key steps of this technique are:

- identify test-relevant aspects (something that affects functionality) – referred to as classifications
- identify the corresponding values, using techniques such as boundary value analysis

Skip to main content

- combine different classes from all classifications into test cases.



Thoughts on this page?

